LINUX ASSIGNMENT

Aditi Biju

261100690027

Web Log Analysis using Real Apache Access Logs

Instead of manually creating access.log, actual Apache access logs are generated from a local web server running inside the Kali Linux VM. The log records are produced automatically when HTTP requests are sent using curl, which makes the practical more realistic.

1. Install and start Apache

```
sudo apt update
sudo apt install apache2 curl -y
sudo systemctl start apache2
sudo systemctl status apache2
curl http://127.0.0.1
```

2. Generate actual web server requests

```
mkdir -p ~/web_log_analysis/output
cd ~/web_log_analysis
```

A. Normal website requests

```
curl -s -o /dev/null http://127.0.0.1/
curl -s -o /dev/null http://127.0.0.1/index.html
curl -s -o /dev/null http://127.0.0.1/about
```

B. Requests for unavailable pages

```
curl -s -o /dev/null http://127.0.0.1/admin
curl -s -o /dev/null http://127.0.0.1/backup.zip
curl -s -o /dev/null http://127.0.0.1/wp-login.php
```

C. Directory traversal test pattern

```
curl --path-as-is -s -o /dev/null \
http://127.0.0.1/../../etc/passwd
```

D. Possible SQL injection pattern

```
curl -s -o /dev/null --get \
--data-urlencode "q=union select" \
http://127.0.0.1/search
```

E. Possible XSS pattern

```
curl -s -o /dev/null --get \
--data-urlencode "q=<script>alert(1)</script>" \
http://127.0.0.1/search
```

These are harmless test requests to the local server. They do not execute SQL or JavaScript on the server.

3. Collect the actual Apache logs

Apache normally stores its access logs here:

/var/log/apache2/access.log

First, record the current number of log lines:

```
BEFORE=$(sudo wc -l < /var/log/apache2/access.log)
```

Now run all the requests from Step 2 again. After that, collect only the newly generated records:

```
sudo tail -n +$((BEFORE + 1)) \
/var/log/apache2/access.log > access.log
```

Check the log:

```
cat access.log
```

4. Improve your AWK script

```
nano analyze.awk
```

```
BEGIN {
   print "WEB SERVER SECURITY ANALYSIS"
   print "-----------------------------"
}
{
   request = tolower($0)
   if (request ~ /(\.\.\/|union%20select|<script>|%3cscript%3e|wp-login|\/admin|backup\.zip)/) {
      count[$1]++
      total++
   }
}
END {
   print "\nSuspicious requests by IP address:"
   for (ip in count) {
      printf "%-18s %d requests", ip, count[ip]
      if (count[ip] >= 3)
         print " - SUSPICIOUS"
      else
         print " - Below threshold"
```

```
    }
    print "\nTotal suspicious-pattern requests:", total
}
```

5. Update your Bash script

```
awk '$9 == 200 {count++} END {
    print "Successful requests:", count+0
}' access.log
```

And this command to count HTTP errors:

```
awk '$9 >= 400 {count++} END {
    print "Error requests:", count+0
}' access.log
```

6. Check your results

```
chmod +x analyze.sh
./analyze.sh

cat access.log
web_log_analysis/
```

7. Method Used to Obtain the Log File

For this assignment, I used Apache2 in my Kali Linux virtual machine to generate web server access logs. Apache was installed and started locally, and I accessed the server using the curl command.

I generated normal requests as well as test requests for unavailable pages, directory traversal patterns and possible SQL injection and XSS inputs. All requests were sent to the local address 127.0.0.1.

Apache automatically recorded these requests in its access log file. I collected the newly generated records from /var/log/apache2/access.log and saved them as access.log in my project directory.

This method allowed me to work with actual Apache-generated log entries without accessing or testing any external website.